

commands.sql

```
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
85 FROM orders;
86 SELECT order_date, daily_revenue,
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue
88 FROM (
89     SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue
90     FROM orders o
91     JOIN order_items oi ON oi.order_id = o.order_id
92     JOIN products p ON p.product_id = oi.product_id
93     GROUP BY o.order_date
94 ) t
95 ORDER BY order_date;
96 SELECT customer_id, order_id, order_date, prev_date,
97        order_date - prev_date AS days_between
98 FROM (
99     SELECT customer_id, order_id, order_date,
100            LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_da
101            COUNT(*) OVER (PARTITION BY customer_id) AS order_count
102 FROM orders
103 ) t
```

I/O

AI Agent

110 ms

STDIN

Input for the program (Optional)

Output:

2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

(15 rows)

customer_id	order_id	order_date	prev_date	days_between
1	3	2026-01-15	2026-01-05	
1	8	2026-02-25	2026-01-15	
1	11	2026-03-20	2026-02-25	
2	6	2026-02-10	2026-01-08	
2	12	2026-04-01	2026-02-10	
3	9	2026-03-03	2026-01-20	
3	14	2026-04-18	2026-03-03	
4	10	2026-03-12	2026-02-02	
4	15	2026-05-02	2026-03-12	
5	13	2026-04-09	2026-02-14	

(10 rows)

commands.sql

+

```
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
85 FROM orders;
86 SELECT order_date, daily_revenue,
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue
88 FROM (
89     SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue
90     FROM orders o
91     JOIN order_items oi ON oi.order_id = o.order_id
92     JOIN products p ON p.product_id = oi.product_id
93     GROUP BY o.order_date
94 ) t
95 ORDER BY order_date;
96 SELECT customer_id, order_id, order_date, prev_date,
97        order_date - prev_date AS days_between
98 FROM (
99     SELECT customer_id, order_id, order_date,
100           LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_date,
101           COUNT(*) OVER (PARTITION BY customer_id) AS order_count
102 FROM orders
103 ) +
```

I/O

AI Agent

110 ms

STDIN

Input for the program (Optional)

Output:

2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

rows)

customer_id	order_id	order_date	prev_date	days_between
1	3	2026-01-15	2026-01-05	10
1	8	2026-02-25	2026-01-15	41
1	11	2026-03-20	2026-02-25	23
2	6	2026-02-10	2026-01-08	33
2	12	2026-04-01	2026-02-10	50
3	9	2026-03-03	2026-01-20	42
3	14	2026-04-18	2026-03-03	46
4	10	2026-03-12	2026-02-02	38
4	15	2026-05-02	2026-03-12	51
5	13	2026-04-09	2026-02-14	54

rows)

4		Maize Flour 2kg		Grains		3.50		2
4		Sugar 1kg		Pantry		1.90		3
5		Milk 1L		Dairy		1.50		2
5		Orange Juice		Beverages		2.80		2
6		Rice 5kg		Grains		8.00		2
6		Cooking Oil 1L		Pantry		4.00		1
7		Bottled Water		Beverages		0.80		10
7		Yogurt		Dairy		1.20		4
8		Sugar 1kg		Pantry		1.90		2
8		Milk 1L		Dairy		1.50		3
9		Orange Juice		Beverages		2.80		4
9		Maize Flour 2kg		Grains		3.50		1
10		Cooking Oil 1L		Pantry		4.00		3
10		Bottled Water		Beverages		0.80		12
11		Rice 5kg		Grains		8.00		1
12		Yogurt		Dairy		1.20		6
13		Milk 1L		Dairy		1.50		5
14		Sugar 1kg		Pantry		1.90		4
15		Orange Juice		Beverages		2.80		3

(25 rows)

order_id	product_name	category	price	quantity
1	Milk 1L	Dairy	1.50	4
1	Rice 5kg	Grains	8.00	1
2	Orange Juice	Beverages	2.80	3
2	Bottled Water	Beverages	0.80	6
3	Yogurt	Dairy	1.20	5
3	Cooking Oil 1L	Pantry	4.00	2
4	Maize Flour 2kg	Grains	3.50	2
4	Sugar 1kg	Pantry	1.90	3
5	Milk 1L	Dairy	1.50	2
5	Orange Juice	Beverages	2.80	2
6	Rice 5kg	Grains	8.00	2
6	Cooking Oil 1L	Pantry	4.00	1
7	Bottled Water	Beverages	0.80	10
7	Yogurt	Dairy	1.20	4
8	Sugar 1kg	Pantry	1.90	2
8	Milk 1L	Dairy	1.50	3
9	Orange Juice	Beverages	2.80	4
9	Maize Flour 2kg	Grains	3.50	1

commands.sql

+

```

29 (5, 'Diane Umutoni', 'diane@mail.com', 'Rubavu'),
30 (6, 'Joseph Habimana', 'joseph@mail.com', 'Kigali');
31 INSERT INTO products VALUES
32 (1, 'Milk 1L', 'Dairy', 1.50),
33 (2, 'Yogurt', 'Dairy', 1.20),
34 (3, 'Rice 5kg', 'Grains', 8.00),
35 (4, 'Maize Flour 2kg', 'Grains', 3.50),
36 (5, 'Orange Juice', 'Beverages', 2.80),
37 (6, 'Bottled Water', 'Beverages', 0.80),
38 (7, 'Cooking Oil 1L', 'Pantry', 4.00),
39 (8, 'Sugar 1kg', 'Pantry', 1.90);
40 INSERT INTO orders VALUES
41 (1, 1, '2026-01-05'), (2, 2, '2026-01-08'), (3, 1, '2026-01-15'),
42 (4, 3, '2026-01-20'), (5, 4, '2026-02-02'), (6, 2, '2026-02-10'),
43 (7, 5, '2026-02-14'), (8, 1, '2026-02-25'), (9, 3, '2026-03-03'),
44 (10, 4, '2026-03-12'), (11, 1, '2026-03-20'), (12, 2, '2026-04-01'),
45 (13, 5, '2026-04-09'), (14, 3, '2026-04-18'), (15, 4, '2026-05-02');
46 INSERT INTO order_items VALUES
47 (1, 1, 1, 4), (2, 1, 3, 1), (3, 2, 5, 3), (4, 2, 6, 6), (5, 3, 2, 5),
48 (6, 3, 7, 2), (7, 4, 4, 2), (8, 4, 8, 3), (9, 5, 1, 2), (10, 5, 5, 2),
49 (11, 6, 3, 2), (12, 6, 7, 1), (13, 7, 6, 10), (14, 7, 2, 4), (15, 8, 8, 2),
50 (16, 8, 1, 3), (17, 9, 5, 4), (18, 9, 4, 1), (19, 10, 7, 3), (20, 10, 6, 12),
51 (21, 11, 3, 1), (22, 12, 2, 6), (23, 13, 1, 5), (24, 14, 8, 4), (25, 15, 5, 3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id;

```

I/O

AI Agent

106 ms

STDIN

Input for the program (Optional)

Output:

customer_id	customer_name	order_id	order_date
1	Aline Uwase	1	2026-01-05
1	Aline Uwase	3	2026-01-15
1	Aline Uwase	11	2026-03-20
1	Aline Uwase	8	2026-02-25
2	Eric Mugisha	2	2026-01-08
2	Eric Mugisha	12	2026-04-01
2	Eric Mugisha	6	2026-02-10
3	Grace Ineza	9	2026-03-03
3	Grace Ineza	4	2026-01-20
3	Grace Ineza	14	2026-04-18
4	Patrick Nkurunziza	5	2026-02-02
4	Patrick Nkurunziza	15	2026-05-02
4	Patrick Nkurunziza	10	2026-03-12
5	Diane Umutoni	7	2026-02-14
5	Diane Umutoni	13	2026-04-09
6	Joseph Habimana		

(16 rows)



commands.sql

```
43 (7,5,'2026-02-14'),(8,1,'2026-02-25'),(9,3,'2026-03-03'),
44 (10,4,'2026-03-12'),(11,1,'2026-03-20'),(12,2,'2026-04-01'),
45 (13,5,'2026-04-09'),(14,3,'2026-04-18'),(15,4,'2026-05-02');
46 INSERT INTO order_items VALUES
47 (1,1,1,4),(2,1,3,1),(3,2,5,3),(4,2,6,6),(5,3,2,5),
48 (6,3,7,2),(7,4,4,2),(8,4,8,3),(9,5,1,2),(10,5,5,2),
49 (11,6,3,2),(12,6,7,1),(13,7,6,10),(14,7,2,4),(15,8,8,2),
50 (16,8,1,3),(17,9,5,4),(18,9,4,1),(19,10,7,3),(20,10,6,12),
51 (21,11,3,1),(22,12,2,6),(23,13,1,5),(24,14,8,4),(25,15,5,3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
```

I/O

AI Agent

131 ms

STDIN

Input for the program (Optional)

Output:

2	Eric Mugisha		2	2026-01-08
2	Eric Mugisha		12	2026-04-01
2	Eric Mugisha		6	2026-02-10
3	Grace Ineza		9	2026-03-03
3	Grace Ineza		4	2026-01-20
3	Grace Ineza		14	2026-04-18
4	Patrick Nkurunziza		5	2026-02-02
4	Patrick Nkurunziza		15	2026-05-02
4	Patrick Nkurunziza		10	2026-03-12
5	Diane Umutoni		7	2026-02-14
5	Diane Umutoni		13	2026-04-09
6	Joseph Habimana			

(16 rows)

customer_name	total_spend
Patrick Nkurunziza	38.60
Eric Mugisha	40.40
Aline Uwase	44.30

(3 rows)

commands.sql

```
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
```

I/O

AI Agent

130 ms

STDIN

Input for the program (Optional)

Output
(5 rows)

customer_id	order_id	order_date	order_number
1	1	2026-01-05	1
1	3	2026-01-15	2
1	8	2026-02-25	3
1	11	2026-03-20	4
2	2	2026-01-08	1
2	6	2026-02-10	2
2	12	2026-04-01	3
3	4	2026-01-20	1
3	9	2026-03-03	2
3	14	2026-04-18	3
4	5	2026-02-02	1
4	10	2026-03-12	2
4	15	2026-05-02	3
5	7	2026-02-14	1
5	13	2026-04-09	2

(15 rows)

commands.sql

```
63 WITH customer_totals AS (  
64     SELECT c.customer_id, c.customer_name,  
65            SUM(oi.quantity * p.price) AS total_spend  
66     FROM customers c  
67     JOIN orders o ON o.customer_id = c.customer_id  
68     JOIN order_items oi ON oi.order_id = o.order_id  
69     JOIN products p ON p.product_id = oi.product_id  
70     GROUP BY c.customer_id, c.customer_name  
71 )  
72 SELECT customer_name, total_spend  
73 FROM customer_totals  
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);  
75 SELECT c.customer_name,  
76        SUM(oi.quantity * p.price) AS total_spend,  
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank  
78 FROM customers c  
79 JOIN orders o ON o.customer_id = c.customer_id  
80 JOIN order_items oi ON oi.order_id = o.order_id  
81 JOIN products p ON p.product_id = oi.product_id  
82 GROUP BY c.customer_name;  
83 SELECT customer_id, order_id, order_date,  
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number  
85 FROM orders;  
86 SELECT order_date, daily_revenue,  
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue  
88 FROM (  
89     SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue  
90     FROM orders o  
91     JOIN order_items oi ON oi.order_id = o.order_id  
92     JOIN products p ON p.product_id = oi.product_id  
93     GROUP BY o.order_date  
94 )
```

I/O

AI Agent

STDIN

Input for the program (Optional)

Output:

(15 rows)

order_date	daily_revenue	running_revenue
2026-01-05	14.00	14.00
2026-01-08	13.20	27.20
2026-01-15	14.00	41.20
2026-01-20	12.70	53.90
2026-02-02	8.60	62.50
2026-02-10	20.00	82.50
2026-02-14	12.80	95.30
2026-02-25	8.30	103.60
2026-03-03	14.70	118.30
2026-03-12	21.60	139.90
2026-03-20	8.00	147.90
2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

(15 rows)

commands.sql × +

```
50 (16,8,1,3),(17,9,5,4),(18,9,4,1),(19,10,7,3),(20,10,6,12),
51 (21,11,3,1),(22,12,2,6),(23,13,1,5),(24,14,8,4),(25,15,5,3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
```

I/O

AI Agent

138 ms ✓

STDIN

Input for the program (Optional)

Output:

5		Diane Umutoni		7		2026-02-14
5		Diane Umutoni		13		2026-04-09
6		Joseph Habimana				

(16 rows)

customer_name	total_spend
Patrick Nkurunziza	38.60
Eric Mugisha	40.40
Aline Uwase	44.30

(3 rows)

customer_name	total_spend	spend_rank
Aline Uwase	44.30	1
Eric Mugisha	40.40	2
Patrick Nkurunziza	38.60	3
Grace Ineza	35.00	4
Diane Umutoni	20.30	5

(5 rows)

Monthly Status				
Month	Income		Expenses	Net
2026-09	30,000 RWF		10,000 RWF	20,000 RWF
2026-08	466,000 RWF		471,250 RWF	-5,250 RWF
2026-07	376,100 RWF		329,600 RWF	46,500 RWF
2026-06	121,700 RWF		102,000 RWF	19,700 RWF
2026-05	98,500 RWF		163,000 RWF	-64,500 RWF
2026-04	21,300 RWF		143,000 RWF	-121,700 RWF
2026-03	94,000 RWF		69,000 RWF	25,000 RWF

Financial Performance Summary - Q3 2026					
Period		Revenue		Profitability	
Month		Income	Expenses	Net	Margin
2026-09		30,000 RWF	10,000 RWF	20,000 RWF	66.67%
2026-08		466,000 RWF	471,250 RWF	-5,250 RWF	-1.12%
2026-07		376,100 RWF	329,600 RWF	46,500 RWF	12.36%
2026-06		121,700 RWF	102,000 RWF	19,700 RWF	16.19%
2026-05		98,500 RWF	163,000 RWF	-64,500 RWF	-65.58%
2026-04		21,300 RWF	143,000 RWF	-121,700 RWF	-571.36%
2026-03		94,000 RWF	69,000 RWF	25,000 RWF	26.49%

order_id	customer_name	city	order_date
1	Aline Uwase	Kigali	2026-01-05
2	Eric Mugisha	Huye	2026-01-08
3	Aline Uwase	Kigali	2026-01-15
4	Grace Ineza	Musanze	2026-01-20
5	Patrick Nkurunziza	Kigali	2026-02-02
6	Eric Mugisha	Huye	2026-02-10
7	Diane Umutoni	Rubavu	2026-02-14
8	Aline Uwase	Kigali	2026-02-25
9	Grace Ineza	Musanze	2026-03-03
10	Patrick Nkurunziza	Kigali	2026-03-12
11	Aline Uwase	Kigali	2026-03-20
12	Eric Mugisha	Huye	2026-04-01
13	Diane Umutoni	Rubavu	2026-04-09
14	Grace Ineza	Musanze	2026-04-18
15	Patrick Nkurunziza	Kigali	2026-05-02

(15 rows)

order_id	product_name	category	price	quantity
1	Milk 1L	Dairy	1.50	4
1	Rice 5kg	Grains	8.00	1
2	Orange Juice	Beverages	2.80	3
2	Bottled Water	Beverages	0.80	6
3	Yogurt	Dairy	1.20	5
3	Cooking Oil 1L	Pantry	4.00	2
4	Maize Flour 2kg	Grains	3.50	2
4	Sugar 1kg	Pantry	1.90	3
5	Milk 1L	Dairy	1.50	2
5	Orange Juice	Beverages	2.80	2
6	Rice 5kg	Grains	8.00	2
6	Cooking Oil 1L	Pantry	4.00	1
7	Bottled Water	Beverages	0.80	10
7	Yogurt	Dairy	1.20	4
8	Sugar 1kg	Pantry	1.90	2
8	Milk 1L	Dairy	1.50	3
9	Orange Juice	Beverages	2.80	4
9	Maize Flour 2kg	Grains	3.50	1

4	Maize Flour 2kg	Grains	3.50	2
4	Sugar 1kg	Pantry	1.90	3
5	Milk 1L	Dairy	1.50	2
5	Orange Juice	Beverages	2.80	2
6	Rice 5kg	Grains	8.00	2
6	Cooking Oil 1L	Pantry	4.00	1
7	Bottled Water	Beverages	0.80	10
7	Yogurt	Dairy	1.20	4
8	Sugar 1kg	Pantry	1.90	2
8	Milk 1L	Dairy	1.50	3
9	Orange Juice	Beverages	2.80	4
9	Maize Flour 2kg	Grains	3.50	1
10	Cooking Oil 1L	Pantry	4.00	3
10	Bottled Water	Beverages	0.80	12
11	Rice 5kg	Grains	8.00	1
12	Yogurt	Dairy	1.20	6
13	Milk 1L	Dairy	1.50	5
14	Sugar 1kg	Pantry	1.90	4
15	Orange Juice	Beverages	2.80	3

rows)

4		Maize Flour 2kg		Grains		3.50		2
4		Sugar 1kg		Pantry		1.90		3
5		Milk 1L		Dairy		1.50		2
5		Orange Juice		Beverages		2.80		2
6		Rice 5kg		Grains		8.00		2
6		Cooking Oil 1L		Pantry		4.00		1
7		Bottled Water		Beverages		0.80		10
7		Yogurt		Dairy		1.20		4
8		Sugar 1kg		Pantry		1.90		2
8		Milk 1L		Dairy		1.50		3
9		Orange Juice		Beverages		2.80		4
9		Maize Flour 2kg		Grains		3.50		1
10		Cooking Oil 1L		Pantry		4.00		3
10		Bottled Water		Beverages		0.80		12
11		Rice 5kg		Grains		8.00		1
12		Yogurt		Dairy		1.20		6
13		Milk 1L		Dairy		1.50		5
14		Sugar 1kg		Pantry		1.90		4
15		Orange Juice		Beverages		2.80		3

(25 rows)

commands.sql

+

```
29 (5, 'Diane Umutoni', 'diane@mail.com', 'Rubavu'),
30 (6, 'Joseph Habimana', 'joseph@mail.com', 'Kigali');
31 INSERT INTO products VALUES
32 (1, 'Milk 1L', 'Dairy', 1.50),
33 (2, 'Yogurt', 'Dairy', 1.20),
34 (3, 'Rice 5kg', 'Grains', 8.00),
35 (4, 'Maize Flour 2kg', 'Grains', 3.50),
36 (5, 'Orange Juice', 'Beverages', 2.80),
37 (6, 'Bottled Water', 'Beverages', 0.80),
38 (7, 'Cooking Oil 1L', 'Pantry', 4.00),
39 (8, 'Sugar 1kg', 'Pantry', 1.90);
40 INSERT INTO orders VALUES
41 (1, 1, '2026-01-05'), (2, 2, '2026-01-08'), (3, 1, '2026-01-15'),
42 (4, 3, '2026-01-20'), (5, 4, '2026-02-02'), (6, 2, '2026-02-10'),
43 (7, 5, '2026-02-14'), (8, 1, '2026-02-25'), (9, 3, '2026-03-03'),
44 (10, 4, '2026-03-12'), (11, 1, '2026-03-20'), (12, 2, '2026-04-01'),
45 (13, 5, '2026-04-09'), (14, 3, '2026-04-18'), (15, 4, '2026-05-02');
46 INSERT INTO order_items VALUES
47 (1, 1, 1, 4), (2, 1, 3, 1), (3, 2, 5, 3), (4, 2, 6, 6), (5, 3, 2, 5),
48 (6, 3, 7, 2), (7, 4, 4, 2), (8, 4, 8, 3), (9, 5, 1, 2), (10, 5, 5, 2),
49 (11, 6, 3, 2), (12, 6, 7, 1), (13, 7, 6, 10), (14, 7, 2, 4), (15, 8, 8, 2),
50 (16, 8, 1, 3), (17, 9, 5, 4), (18, 9, 4, 1), (19, 10, 7, 3), (20, 10, 6, 12),
51 (21, 11, 3, 1), (22, 12, 2, 6), (23, 13, 1, 5), (24, 14, 8, 4), (25, 15, 5, 3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

I/O

AI Agent

106 ms

STDIN

Input for the program (Optional)

Output:

customer_id	customer_name	order_id	order_date
1	Aline Uwase	1	2026-01-05
1	Aline Uwase	3	2026-01-15
1	Aline Uwase	11	2026-03-20
1	Aline Uwase	8	2026-02-25
2	Eric Mugisha	2	2026-01-08
2	Eric Mugisha	12	2026-04-01
2	Eric Mugisha	6	2026-02-10
3	Grace Ineza	9	2026-03-03
3	Grace Ineza	4	2026-01-20
3	Grace Ineza	14	2026-04-18
4	Patrick Nkurunziza	5	2026-02-02
4	Patrick Nkurunziza	15	2026-05-02
4	Patrick Nkurunziza	10	2026-03-12
5	Diane Umutoni	7	2026-02-14
5	Diane Umutoni	13	2026-04-09
6	Joseph Habimana		

(16 rows)



commands.sql

```
43 (7,5,'2026-02-14'),(8,1,'2026-02-25'),(9,3,'2026-03-03'),
44 (10,4,'2026-03-12'),(11,1,'2026-03-20'),(12,2,'2026-04-01'),
45 (13,5,'2026-04-09'),(14,3,'2026-04-18'),(15,4,'2026-05-02');
46 INSERT INTO order_items VALUES
47 (1,1,1,4),(2,1,3,1),(3,2,5,3),(4,2,6,6),(5,3,2,5),
48 (6,3,7,2),(7,4,4,2),(8,4,8,3),(9,5,1,2),(10,5,5,2),
49 (11,6,3,2),(12,6,7,1),(13,7,6,10),(14,7,2,4),(15,8,8,2),
50 (16,8,1,3),(17,9,5,4),(18,9,4,1),(19,10,7,3),(20,10,6,12),
51 (21,11,3,1),(22,12,2,6),(23,13,1,5),(24,14,8,4),(25,15,5,3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
```

I/O

AI Agent

131 ms

STDIN

Input for the program (Optional)

Output:

2		Eric Mugisha		2		2026-01-08
2		Eric Mugisha		12		2026-04-01
2		Eric Mugisha		6		2026-02-10
3		Grace Ineza		9		2026-03-03
3		Grace Ineza		4		2026-01-20
3		Grace Ineza		14		2026-04-18
4		Patrick Nkurunziza		5		2026-02-02
4		Patrick Nkurunziza		15		2026-05-02
4		Patrick Nkurunziza		10		2026-03-12
5		Diane Umutoni		7		2026-02-14
5		Diane Umutoni		13		2026-04-09
6		Joseph Habimana				

(16 rows)

customer_name	total_spend
Patrick Nkurunziza	38.60
Eric Mugisha	40.40
Aline Uwase	44.30

(3 rows)



commands.sql × +

```
50 (16,8,1,3),(17,9,5,4),(18,9,4,1),(19,10,7,3),(20,10,6,12),
51 (21,11,3,1),(22,12,2,6),(23,13,1,5),(24,14,8,4),(25,15,5,3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
```

I/O

AI Agent

138 ms ✓

STDIN

Input for the program (Optional)

Output:

5	Diane Umutoni	7	2026-02-14
5	Diane Umutoni	13	2026-04-09
6	Joseph Habimana		

(16 rows)

customer_name	total_spend
Patrick Nkurunziza	38.60
Eric Mugisha	40.40
Aline Uwase	44.30

(3 rows)

customer_name	total_spend	spend_rank
Aline Uwase	44.30	1
Eric Mugisha	40.40	2
Patrick Nkurunziza	38.60	3
Grace Ineza	35.00	4
Diane Umutoni	20.30	5

(5 rows)

commands.sql × +

```
50 (16,8,1,3),(17,9,5,4),(18,9,4,1),(19,10,7,3),(20,10,6,12),
51 (21,11,3,1),(22,12,2,6),(23,13,1,5),(24,14,8,4),(25,15,5,3);
52 SELECT o.order_id, c.customer_name, c.city, o.order_date
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
```

I/O

AI Agent

138 ms ✓

STDIN

Input for the program (Optional)

Output:

5	Diane Umutoni		7	2026-02-14
5	Diane Umutoni		13	2026-04-09
6	Joseph Habimana			

(16 rows)

customer_name	total_spend
Patrick Nkurunziza	38.60
Eric Mugisha	40.40
Aline Uwase	44.30

(3 rows)

customer_name	total_spend	spend_rank
Aline Uwase	44.30	1
Eric Mugisha	40.40	2
Patrick Nkurunziza	38.60	3
Grace Ineza	35.00	4
Diane Umutoni	20.30	5

(5 rows)

commands.sql ×

+

```
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65           SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
```

commands.sql

```
53 FROM orders o
54 INNER JOIN customers c ON o.customer_id = c.customer_id;
55 SELECT oi.order_id, p.product_name, p.category, p.price, oi.quantity
56 FROM order_items oi
57 JOIN products p ON oi.product_id = p.product_id;
58 SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
59 FROM customers c
60 LEFT JOIN orders o ON c.customer_id = o.customer_id
61 ORDER BY c.customer_id;
62 -- CTE
63 WITH customer_totals AS (
64     SELECT c.customer_id, c.customer_name,
65            SUM(oi.quantity * p.price) AS total_spend
66     FROM customers c
67     JOIN orders o ON o.customer_id = c.customer_id
68     JOIN order_items oi ON oi.order_id = o.order_id
69     JOIN products p ON p.product_id = oi.product_id
70     GROUP BY c.customer_id, c.customer_name
71 )
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
```

I/O

AI Agent

130 ms

STDIN

Input for the program (Optional)

Output
(5 rows)

customer_id	order_id	order_date	order_number
1	1	2026-01-05	1
1	3	2026-01-15	2
1	8	2026-02-25	3
1	11	2026-03-20	4
2	2	2026-01-08	1
2	6	2026-02-10	2
2	12	2026-04-01	3
3	4	2026-01-20	1
3	9	2026-03-03	2
3	14	2026-04-18	3
4	5	2026-02-02	1
4	10	2026-03-12	2
4	15	2026-05-02	3
5	7	2026-02-14	1
5	13	2026-04-09	2

(15 rows)

commands.sql

+

```
63 WITH customer_totals AS (  
64     SELECT c.customer_id, c.customer_name,  
65            SUM(oi.quantity * p.price) AS total_spend  
66     FROM customers c  
67     JOIN orders o ON o.customer_id = c.customer_id  
68     JOIN order_items oi ON oi.order_id = o.order_id  
69     JOIN products p ON p.product_id = oi.product_id  
70     GROUP BY c.customer_id, c.customer_name  
71 )  
72 SELECT customer_name, total_spend  
73 FROM customer_totals  
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);  
75 SELECT c.customer_name,  
76        SUM(oi.quantity * p.price) AS total_spend,  
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank  
78 FROM customers c  
79 JOIN orders o ON o.customer_id = c.customer_id  
80 JOIN order_items oi ON oi.order_id = o.order_id  
81 JOIN products p ON p.product_id = oi.product_id  
82 GROUP BY c.customer_name;  
83 SELECT customer_id, order_id, order_date,  
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number  
85 FROM orders;  
86 SELECT order_date, daily_revenue,  
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue  
88 FROM (  
89     SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue  
90     FROM orders o  
91     JOIN order_items oi ON oi.order_id = o.order_id  
92     JOIN products p ON p.product_id = oi.product_id  
93     GROUP BY o.order_date  
94 )
```

I/O

AI Agent

116 ms

STDIN

Input for the program (Optional)

Output:

(15 rows)

order_date	daily_revenue	running_revenue
2026-01-05	14.00	14.00
2026-01-08	13.20	27.20
2026-01-15	14.00	41.20
2026-01-20	12.70	53.90
2026-02-02	8.60	62.50
2026-02-10	20.00	82.50
2026-02-14	12.80	95.30
2026-02-25	8.30	103.60
2026-03-03	14.70	118.30
2026-03-12	21.60	139.90
2026-03-20	8.00	147.90
2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

(15 rows)

commands.sql

```
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
85 FROM orders;
86 SELECT order_date, daily_revenue,
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue
88 FROM (
89   SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue
90   FROM orders o
91   JOIN order_items oi ON oi.order_id = o.order_id
92   JOIN products p ON p.product_id = oi.product_id
93   GROUP BY o.order_date
94 ) t
95 ORDER BY order_date;
96 SELECT customer_id, order_id, order_date, prev_date,
97        order_date - prev_date AS days_between
98 FROM (
99   SELECT customer_id, order_id, order_date,
100          LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_da
101          COUNT(*) OVER (PARTITION BY customer_id) AS order_count
102 FROM orders
103 ) t
```

I/O

AI Agent

110 ms

STDIN

Input for the program (Optional)

Output:

2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

(15 rows)

customer_id	order_id	order_date	prev_date	days_between
1	3	2026-01-15	2026-01-05	
1	8	2026-02-25	2026-01-15	
1	11	2026-03-20	2026-02-25	
2	6	2026-02-10	2026-01-08	
2	12	2026-04-01	2026-02-10	
3	9	2026-03-03	2026-01-20	
3	14	2026-04-18	2026-03-03	
4	10	2026-03-12	2026-02-02	
4	15	2026-05-02	2026-03-12	
5	13	2026-04-09	2026-02-14	

(10 rows)



commands.sql

+

```
72 SELECT customer_name, total_spend
73 FROM customer_totals
74 WHERE total_spend > (SELECT AVG(total_spend) FROM customer_totals);
75 SELECT c.customer_name,
76        SUM(oi.quantity * p.price) AS total_spend,
77        RANK() OVER (ORDER BY SUM(oi.quantity * p.price) DESC) AS spend_rank
78 FROM customers c
79 JOIN orders o ON o.customer_id = c.customer_id
80 JOIN order_items oi ON oi.order_id = o.order_id
81 JOIN products p ON p.product_id = oi.product_id
82 GROUP BY c.customer_name;
83 SELECT customer_id, order_id, order_date,
84        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS order_number
85 FROM orders;
86 SELECT order_date, daily_revenue,
87        SUM(daily_revenue) OVER (ORDER BY order_date) AS running_revenue
88 FROM (
89     SELECT o.order_date, SUM(oi.quantity * p.price) AS daily_revenue
90     FROM orders o
91     JOIN order_items oi ON oi.order_id = o.order_id
92     JOIN products p ON p.product_id = oi.product_id
93     GROUP BY o.order_date
94 ) t
95 ORDER BY order_date;
96 SELECT customer_id, order_id, order_date, prev_date,
97        order_date - prev_date AS days_between
98 FROM (
99     SELECT customer_id, order_id, order_date,
100           LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_date,
101           COUNT(*) OVER (PARTITION BY customer_id) AS order_count
102 FROM orders
103 ) +
```

I/O

AI Agent

110 ms

STDIN

Input for the program (Optional)

Output:

2026-04-01	7.20	155.10
2026-04-09	7.50	162.60
2026-04-18	7.60	170.20
2026-05-02	8.40	178.60

rows)

customer_id	order_id	order_date	prev_date	days_between
1	3	2026-01-15	2026-01-05	10
1	8	2026-02-25	2026-01-15	41
1	11	2026-03-20	2026-02-25	23
2	6	2026-02-10	2026-01-08	33
2	12	2026-04-01	2026-02-10	50
3	9	2026-03-03	2026-01-20	42
3	14	2026-04-18	2026-03-03	46
4	10	2026-03-12	2026-02-02	38
4	15	2026-05-02	2026-03-12	51
5	13	2026-04-09	2026-02-14	54

rows)

order_id	customer_name	city	order_date
1	Aline Uwase	Kigali	2026-01-05
2	Eric Mugisha	Huye	2026-01-08
3	Aline Uwase	Kigali	2026-01-15
4	Grace Ineza	Musanze	2026-01-20
5	Patrick Nkurunziza	Kigali	2026-02-02
6	Eric Mugisha	Huye	2026-02-10
7	Diane Umutoni	Rubavu	2026-02-14
8	Aline Uwase	Kigali	2026-02-25
9	Grace Ineza	Musanze	2026-03-03
10	Patrick Nkurunziza	Kigali	2026-03-12
11	Aline Uwase	Kigali	2026-03-20
12	Eric Mugisha	Huye	2026-04-01
13	Diane Umutoni	Rubavu	2026-04-09
14	Grace Ineza	Musanze	2026-04-18
15	Patrick Nkurunziza	Kigali	2026-05-02

(15 rows)